A Comparative Survey of Model-Based and Model-Free Reinforcement Learning

Nihal Abdul Naseer, Mohammed Alawadhi, and Abdulhafedh Al-Zubedi
American University of Sharjah. United Arab Emirates
{b00112155, b00108492, b00112488}@aus.edu

***Abstract*—Reinforcement learning (RL) has emerged as a principal framework for sequential decision-making under uncertainty, with applications spanning games, robotics, healthcare, autonomous driving, and industrial operations. Within this framework, two paradigms dominate: *model-based* methods, which learn or assume an explicit model of the environment and use it for planning, and *model-free* methods, which estimate values or policies directly from sampled interaction. This paper presents a comparative survey of the two families, organised around the Markov Decision Process formalism that both share. We review the principal algorithms in each paradigm—from value and policy iteration through modern actor-critic and deep variants—and compare them along nine dimensions including sample efficiency, asymptotic performance, robustness to model bias, and computational cost. We also survey representative applications across five domains and discuss open challenges in scalability, safety, and real-world deployment. The central observation of the survey is that the trade-off between the two paradigms originates directly in the structure of the Bellman optimality equation, and that the most recent literature increasingly favours hybrid methods that combine planning with direct policy learning. The choice of paradigm, we argue, is driven less by any specific task than by the cost of environment interaction and the availability of an accurate forward model.**

***Index Terms*—Reinforcement learning, model-based reinforcement learning, model-free reinforcement learning, Markov decision process, deep reinforcement learning, policy optimization, survey.**

I. Introduction

SEQUENTIAL decision-making under uncertainty is a central challenge in artificial intelligence, arising whenever an agent must choose actions whose consequences unfold over time and whose optimal choice depends on delayed, stochastic feedback. Reinforcement learning (RL) provides a principled computational framework for this problem: an agent interacts with an environment, observes rewards contingent on its actions, and incrementally refines its behaviour so as to maximise a long-run measure of cumulative reward [11], [32]. Over the past decade, RL has delivered landmark results in domains ranging from superhuman game-playing [8], [12] and dexterous robotic manipulation [4], [9] to clinical decision support [36] and autonomous driving [37]. These advances have been driven not by a single algorithm but by a broad and increasingly diverse algorithmic taxonomy.

The most consequential axis of this taxonomy is the distinction between *model-based* and *model-free* methods. Model-based reinforcement learning either assumes or learns an explicit representation of the environment's transition dynamics and uses this model to plan ahead [31]. Model-free reinforcement learning, by contrast, sidesteps the model entirely, estimating values or policies directly from sampled interaction [33]. The two families have developed largely in parallel, each producing its own characteristic algorithms, its own theoretical guarantees, and its own empirical strengths and weaknesses. Yet practitioners repeatedly face the same question when applying RL to a new problem: which paradigm should I choose, and on what basis?

Despite an extensive literature on each family individually, comparative surveys that treat the two paradigms symmetrically remain scarce, and most focus on a single application domain or a narrow pair of algorithms. Recent comprehensive surveys have emphasised model-based methods specifically [31], deep reinforcement learning broadly [33], or particular algorithmic taxonomies [34], leaving a gap for a work that draws out the underlying source of their trade-offs. This paper addresses that gap. Rather than cataloguing algorithms paradigm-by-paradigm, we organise the comparison around the Markov Decision Process formalism that both families share, showing that the practical differences between them follow directly from a single algorithmic choice: whether to evaluate the Bellman optimality expectation analytically through a model, or to approximate it empirically through samples.

The remainder of the paper is structured as follows. Section II introduces the Markov Decision Process formalism and defines the two paradigms in its terms. Section **??** surveys model-based reinforcement learning, covering value-function approximation, transition-model learning, and planning algorithms including dynamic programming, model-predictive control, and meta-policy optimisation. Section **??** surveys model-free methods, including the foundational temporal-difference algorithms, deep value-based methods such as DQN and Rainbow, policy-gradient methods such as TRPO and PPO, and actor-critic methods such as DDPG, TD3, and SAC. Section V presents a systematic comparison of the two paradigms across nine dimensions and discusses the hybrid methods that are increasingly blurring the boundary between them. Section VI surveys representative applications across games, robotics, autonomous driving, healthcare, and energy systems. Section VII identifies open research challenges and future directions, and Section VIII concludes.

II. Background: Markov Decision Processes

Reinforcement learning (RL) formalises the problem faced by a goal-directed agent that must learn, through interaction alone, how to act in an environment so as to maximise a long-run scalar reward [11], [32]. Unlike supervised learning, the agent is never told which action is correct; feedback arrives only as a reward signal that may be sparse, delayed, and entangled with the agent's own past choices. To reason about this setting mathematically, the field adopts the *Markov Decision Process* (MDP) as its unifying abstraction [11], [31]. The MDP framework underpins both model-based and model-free methods surveyed in this paper; the distinction between the two paradigms lies entirely in how they exploit, or avoid, the components of this shared formalism.

A. The Markov Decision Process

An MDP is specified by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state-transition probability function, $R : \mathcal{S} \times \mathcal{A} \to R$ is the reward function, and $\gamma \in [0, 1)$ is a discount factor. At each discrete time step t, the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives an immediate reward $r_{t+1} = R(s_t, a_t)$, and transitions to a next state s_{t+1} drawn from $P(\cdot \mid s_t, a_t)$.

The defining assumption is the *Markov property*: the distribution of the next state and reward depends only on the current state and action, not on the trajectory that led to s_t,

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t). \tag{1}$$

This assumption is what permits the recursive decomposition of long-term values into one-step updates that makes RL computationally tractable.

B. Policies, Returns, and Value Functions

A *policy* π is a mapping from states to a probability distribution over actions, $\pi(a \mid s) = \Pr(A_t = a \mid S_t = s)$; deterministic policies are the special case in which probability mass is concentrated on a single action. The agent's objective is to find a policy that maximises the expected discounted *return*,

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1}, \tag{2}$$

where the discount factor γ both ensures convergence over infinite horizons and encodes a preference for near-term reward.

Two value functions are central to almost every RL algorithm. The *state-value function* under policy π is the expected return obtained when starting from state s and following π thereafter,

$$V^{\pi}(s) = E_{\pi}[\, G_t \mid S_t = s \,], \tag{3}$$

while the *action-value function*, or Q-function, is the expected return conditioned additionally on an initial action a,

$$Q^{\pi}(s, a) = E_{\pi}[\, G_t \mid S_t = s, A_t = a \,]. \tag{4}$$

The Q-function is particularly important in model-free methods, since a greedy action $\arg\max_a Q(s, a)$ can be selected directly without requiring knowledge of P [11].

C. Bellman Equations and Optimality

Both value functions satisfy recursive consistency conditions known as the *Bellman equations*. For any policy π,

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \, [\, R(s, a) + \gamma V^{\pi}(s') \,]. \tag{5}$$

The goal of solving an MDP is to obtain an *optimal policy* π^* whose value dominates that of every other policy at every state. Its value function $V^*(s) = \max_{\pi} V^{\pi}(s)$ satisfies the *Bellman optimality equation*,

$$V^*(s) = \max_{a} \sum_{s'} P(s' \mid s, a) \, [\, R(s, a) + \gamma V^*(s') \,], \tag{6}$$

with an analogous form holding for $Q^*(s, a)$. An optimal policy can then be recovered greedily as $\pi^*(s) = \arg\max_a Q^*(s, a)$.

D. Two Routes to the Optimal Policy

Equation (6) also exposes the branching point that organises the rest of this survey. The right-hand side evaluates an expectation under the transition function P together with the reward function R — that is, under an explicit *model* of the environment. Whether the agent has access to this model, or must operate without it, gives rise to two fundamentally different strategies for solving MDPs [32], [31]. *Model-based* methods either assume or learn an explicit representation of P and R, and then use planning algorithms such as dynamic programming or model-predictive control to exploit them (Section **??**). *Model-free* methods, by contrast, sidestep the model entirely: they estimate V^{π} or Q^{π} directly from sampled transitions, using only observed tuples $(s_t, a_t, r_{t+1}, s_{t+1})$ produced by interaction with the environment (Section **??**). Each family trades off sample efficiency, computational cost, and robustness in distinct ways; the sections that follow examine each in turn before returning, in Section V, to a systematic comparison of the two.

III. Model-based reinforcement learning

Model-based reinforcement learning is the process of indirectly learning optimal behavior by performing actions and observing the results, which include the subsequent state and the immediate reward. The model is then used to find the optimal policy based on the model to get the maximum cumulative reward.

In model-based approaches, the agent first learns a transition model of the environment and then uses planning techniques, such as simulation or dynamic programming, to evaluate and improve its policy. The major benefit of using this approach is the improvement in data efficiency, since it eliminates the need to conduct many real-world experiments. On the other hand, this method is vulnerable to the problem of model bias, where mistakes in the model will lead to poor performance.

A. Value Function methods

A technique wherein the actual value of the value function is approximated by a specific function (such as a linear or nonlinear function) based on a limited subset of the training sample, referred to as function approximation (Xian & Yongchun, 2018).In model-based reinforcement learning, this can be categorized into two mathematical approaches:

1) Parameterized Value Function Approximation:: The parameterized value function approximation approach approximates the value function using a vector of parameters or weights (w). The primary goal of using parameters is to transition from tabular storage to a functional representation, enabling generalization to large-scale space or continuous state space. This reduces the memory requirement significantly, allowing the agent to estimate values for unseen states based on learned properties of similar states.

$$\overline{VE}(w) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, w) \right]^2 \tag{7}$$

The Mean Squared Value Error is a natural objective function that is used to evaluate the approximate function, as established by Sutton and Barto (2018). The most common approach to minimize this error is via Stochastic Gradient Descent (SGD). Where in the context of model-based RL, the agent uses its learnt environment model to generate simulated transitions, updating the parameters as follows:

$$w_{t+1} \doteq w_t + \alpha \left[R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t) \right] \nabla \hat{v}(S_t, w_t) \tag{8}$$

- To execute this approach effectively, specific techniques are used for feature construction and applying approximation to decision-making:
- Coarse Coding: Refers to a linear feature construction technique where states are represented by overlapping receptive fields. Which help to achieve spatial generalization, in which changes in the weight of one state affect multiple states.
- Action value Function: In terms of control, to approximate action values $\hat{q}(s, a, w)$, the action value approximator which uses a parameter vector **w** tp estimate the true action value function $Q(s, a)$ is used. Enabling the agent to choose actions that maximize the predicted future return.
- Deep Value Function Approximation The evolution of function approximation has led from linear methods to the use of Deep Neural Networks. When a neural network is used as the approximator, the method is known as Deep Q-Learning. In this setup, the parameters (w) represent the weights of the network, which are trained to minimize the TD error between the current estimate and a target derived from the Bellman equation:

$$\text{Target} = R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, w_t) \tag{9}$$

The transition from classic to modern function approximators has significantly expanded the capabilities of RL agents. While early methods like the Cerebellar Model Articulation Controller (CMAC) provided stable local approximation for robotics, they lacked the speed required for efficient online learning. In contrast, modern Deep Neural Networks offer superior generalization in high-dimensional spaces; however, they introduce new complexities. According to Mnih et al. (2015), these challenges arise due to the presence of correlation between observations and high variance of updates. To deal with such problems, techniques like Experience Replay and Target Networks have been proposed. According to Kamalapurkar et al. (2016), despite these improvements, these types of structures are highly dependent on the setting of their parameters and can easily converge into local minima, causing instability in the planning process within model-based reinforcement learning.

2) Nonparametric Value Function Approximation: The term "nonparametric value function approximation" does not refer to a function approximation without parameters. Rather, it refers to a function approximation approach where the number of parameters and base form are variable. It does not assume a fixed functional form (w). It estimates the state value based on similarities with previous reccoded transitions stored in a memory buffer, allowing the model's complexity to grow with the data.

Typical approaches for nonparametric function approximation consist of methods utilizing kernel functions (Hang Li, 2012) and techniques based on Gaussian processes (Rasmussen Williams, 2005).The nonparametric function approximation technique approximates the data set with the training sample N, and each sample becomes part of the function approximation, hence the name "approximation method determined by the samples."

To overcome the scaling limitations of standard nonparametric approaches, current research has focused on Sparse Gaussian Processes and Deep Kernel Learning.

These frameworks use inducing points, which are a smaller, representative subset of data to make calculations faster. They bring down the complexity from $O(N^3)$ to $O(m^2N)$. Here m is much smaller, than N and that is what makes it faster. Moreover, the development of Neural Tangent Kernels (NTK) has offered a connection from the stability properties of kernel techniques to the computational scalability of deep learning architectures. This combination of approaches enables model-based reinforcement learning agents to preserve accurate uncertainty estimates while remaining computationally feasible for real-world applications.

B. TRANSITION MODELS

Transition model is a representation of the dynamics of the environment, as a probability of moving from one state to another given an action. defined as P(ss,a). In general, transition models fall into two categories: stochastic models and deterministic models.

1) Deterministic Model: In a deterministic model, every state-action pair has only one outcome, assuming that there is no possibility of noise and randomness in the environment. This is ideal for fixed rules environments like games and noise-free environments.

2) Stochastic Model: Many environments are naturally stochastic, and sensor noise causes stochasticity to rise in real-world scenarios. Transition dynamics typically incorporate both stochastic and deterministic elements, such as the behavior of another automobile on the road and the trajectory of an object falling owing to gravity.

The stochastic model predicts a distribution of next state probabilities, instead of predicting a specific next state, to better represent real world environments' uncertainty:

$$P(s_{t+1}|s_t, a_t) \tag{10}$$

Thus, the agent can know not only what can happen but also the probability of such an occurrence. It is especially necessary in "safe" reinforcement learning because it stops the agent from being overconfident about its predictions.

To learn these models, several methods are utilized:

- Neural Networks: often utilized because they can approximate complicated, multidimensional dynamics; contemporary models make use of Deep Generative Models like Conditional Variational Autoencoders (CVAEs) to model complicated, multimodal distributions (Moerland et al., 2017).
- Probabilistic Models, such as Gaussian processes, provide uncertainty estimates; however, they suffer from scaling issues. Bayesian Neural Ordinary Differential Equations (ODE-RL) is a modern approach that allows the agent to learn state transitions in continuous time, not just discretely spaced time steps (Yıldız et al., 2021). Which is particularly effective for irregular or noisy data sampling systems.
- Ensemble-based methods improve robustness more by training multiple models and combining their predictions to account for uncertainty and lower model bias. Ensemble learning has proven to be effective in applications like PETS (Chua et al., 2018).

3) Model bias: A key challenge for MBRL is model bias, particularly in early training with limited data sets, inaccurately learnt models, and low data efficiency. Yıldız et al. (2021) differentiate between Aleatoric uncertainty, which refers to intrinsic randomness, and Epistemic uncertainty, which refers to the agent's deficiency of knowledge. If a model can't account for epistemic uncertainty, the agent might be overconfident in its "biased" predictions, which could lead to the development of compounding errors, such that small errors at the early stages of planning create unrealistic future states. The trade-off between model accuracy and data efficiency is still a major issue in model-based reinforcement learning

C. Planning and Policy Optimization Methods in Model-Based Reinforcement Learning

Planning in MBRL uses a transition model to simulate future trajectories and evaluate actions before interacting with the real world. Model-based RL outperforms model-free approaches in terms of data efficiency by using "lookahead" searches. These planning procedures help with a wide range of policy optimization methods, from classic to modern techniques.

1) Dynamic Programming: Dynamic Programming forms the basis of model-based algorithms. The technique involves the estimation of states (Policy Estimation) and then the selection of better policies(Policy Promotion) through iterative evaluation. Policy iteration has the problem of requiring a policy estimate at each iteration, which occurs after the previous iteration. The policy estimation ensures that vk accurately converges to v, but the computation time is very slow. Value iteration on the other hand, reduces time by combining both evaluation and improvement in a single sweep, achieving a much higher convergence speed.

Though conceptually efficient, DP falls prey to the "curse of dimensionality" in higher dimensions; however, this can be addressed by Asynchronous DP by concentrating computational effort on important trajectories by considering state transitions randomly or arbitrarily.

2) Guided Policy Search (GPS): The Guided Policy Search (GPS) was developed to address high-dimensional tasks, as direct policy search requires a large number of samples and often falls into poor local optimum.The approach works effectively for deterministic and linear dynamics, but it is extremely difficult for nonlinear dynamic systems and complex tasks. Nonetheless, the algorithm offers an excellent foundation for further development. Subsequently, various enhanced GPS algorithms are proposed. Including cGPS that introduced iterative fitting local dynamic model and learning time-varying linear gaussian controller which solves the learning problem of gaussian controller under effective unknown dynamic conditions. Dual GPS combined several trajectories to increase policy generalization. Researchers were able to optimize high-dimensional nonlinear policies using neural networks by developing GPS as a supervised learning issue and utilizing approaches such as Bregman ADMM and Mirror Descent. This advancement has resulted in GPS being a useful tool for controlling objects based on vision, which was not feasible with the previous linear method.

3) Model Predictive Control (MPC): Previous strategies concentrated on identifying a global policy offline. Model Predictive Control (MPC), on the other hand, emphasizes online planning. In every step, the agent applies its model in order to simulate potential paths for a finite "receding horizon" and after executing the first action it replans for the next step. Such a feedback cycle makes MPC very robust against environment noise. In many contemporary MBRL methods, MPC is combined with Trajectory Sampling (e.g., the PETS algorithm) to ensure high data efficiency while handling complicated, nonlinear dynamics in real time.

4) Model-Based Meta-Policy-Optimization : To enhance robustness, Clavera et al. (2018) proposed Model-Based Meta-Policy-Optimization (MB-MPO) that tackles the significant problem of model-bias, which occurs when an agent could exploit flaws in its learned dynamics. It uses an ensemble of learned dynamic models to meta-optimize a policy for adaptation in each of the learned models with one policy gradient step.This strategy passes the task of dealing with model inconsistencies to the adaptation phase, ensuring the policy's "plasticity" and robustness against flawed models. Results of the paper show that MB-MPO algorithm is ca-

pable of matching the asymptotic performance of model-free methods while needing substantially less experience, 10 to 100 times less experience, effectively extending the model-based approach to more complex domains and longer horizons.

D. Applications of Model-Based RL

MBRL had a major impact across several disciplines by offering a paradigm for sample-efficient behavior and internal world representation. It is extremely important in areas where real-world data collection is costly, risky, or necessitates complicated reasoning. Applications include strategic gaming; AlphaGo Zero employed Monte Carlo Tree Search to attain superhuman performance, while in robotics, it allows precise manipulation, UAV navigation, and autonomous driving through simulation-based planning. Moreover, it has transformed brain science by providing a computational framework for understanding Parkinson's disease, relating dopaminergic dysfunction to a patient's inability to use a "world model" for reward seeking. By closing the gap between theoretical planning and complex tasks, MBRL is essential to both artificial intelligence and cognitive neuroscience. Additionally, MBRL is used to improve smart grids through the modeling of weather and demand trends. It utilizes MPC to "plan" the electricity distribution several hours ahead of time, saving about 20 percent of costs for utilities through offloading of heavy loads during peak times.

IV. Model-free reinforcement learning

Model-free reinforcement learning methods aim to learn optimal policies directly through interaction with the environment, without requiring explicit knowledge of transition dynamics. Unlike planning-based approaches that rely on a known model of the environment, model-free methods estimate value functions or optimize policies using sampled experience. These approaches have gained significant attention due to their applicability in real-world scenarios where accurate environment models are unavailable or difficult to obtain. Broadly, model-free reinforcement learning can be categorized into value-based, policy-based, and actor-critic methods. In this section, we first discuss foundational approaches such as temporal difference learning, followed by value-based methods including Q-learning and its deep extensions. We then examine policy gradient techniques and actor-critic methods, and conclude with recent advances in model-free reinforcement learning.

A. Foundations: Temporal Difference and Value-Based Learning

Model-free reinforcement learning methods are fundamentally built on the idea of learning directly from experience without requiring an explicit model of the environment. Early approaches such as Monte Carlo methods estimate value functions based on complete episodes, updating predictions only after observing the final outcome. However, this approach can be inefficient in tasks where feedback is delayed. To address this limitation, Temporal Difference Learning was introduced as a more efficient alternative.

1) Temporal Difference Learning: Temporal difference (TD) learning (Sutton, 1988) updates predictions incrementally by comparing successive estimates rather than waiting for a final outcome. At each step, the method adjusts its current prediction based on the difference between the predicted value of the current state and the predicted value of the next state. This allows learning to occur continuously during interaction with the environment. A key contribution of TD learning is its ability to combine ideas from dynamic programming and Monte Carlo methods, providing improved computational efficiency and faster convergence in many settings. However, TD methods can sometimes propagate inaccurate estimates when early predictions are misleading.

2) Q-learning: Building upon TD learning, value-based methods aim to directly estimate the quality of actions through action-value functions. A fundamental algorithm in this category is Q-learning (Watkins and Dayan, 1992), which learns the optimal action-value function by iteratively updating estimates based on observed rewards and the maximum expected future return. Q-learning is an off-policy method, as it updates values using the best possible action regardless of the agent's current behavior policy. Its major contribution lies in demonstrating that optimal policies can be learned purely from interaction with the environment, without requiring a model. Despite its theoretical guarantees of convergence under certain conditions, Q-learning can become unstable or inefficient in large or continuous state spaces.

3) SARSA: An alternative to Q-learning is SARSA (Rummery and Niranjan, 1994), an on-policy method that updates action values based on the actual actions taken by the agent. Unlike Q-learning, which evaluates the optimal policy independently of the agent's behavior, SARSA learns the value of the policy being followed, including exploratory actions. This makes SARSA generally more stable and better suited for environments where cautious or risk-aware behavior is desirable. However, it may converge more slowly to the optimal policy compared to off-policy methods.

Together, TD learning, Q-learning, and SARSA form the foundation of modern model-free reinforcement learning, providing the basis for many advanced algorithms discussed in subsequent sections. While these foundational methods are effective in small-scale settings, they struggle to scale to high-dimensional environments. This limitation motivates the development of more advanced value-based methods using function approximation.

B. Value-Based Methods

Value-based methods extend the principles of temporal difference learning by directly estimating the action-value function, enabling agents to select actions that maximize expected cumulative rewards. While tabular methods such as Q-learning are effective in small, discrete environments, they become impractical in high-dimensional state spaces. To address this limitation, deep learning techniques were introduced, leading to the development of the Deep Q-Network (DQN).

1) Deep Q-Network (DQN): DQN (Mnih et al., 2015) combines reinforcement learning with deep neural networks

to approximate the action-value function directly from raw, high-dimensional inputs such as images. However, naïvely applying neural networks in this setting leads to instability and divergence. To overcome this, DQN introduces two key mechanisms: experience replay, which stores and randomly samples past transitions to break correlations in the data, and a target network, which stabilizes learning by providing fixed targets for a period of time. These innovations enabled DQN to achieve human-level performance on a range of benchmark tasks. Nevertheless, DQN struggles with complex tasks requiring long-term planning and can still exhibit unstable learning behavior.

2) Double DQN: A significant issue with DQN is the overestimation of action values caused by the maximization step in the update rule. This problem is addressed by Double DQN(van Hasselt et al., 2015), which decouples action selection from action evaluation. Specifically, the online network is used to select the best action, while the target network evaluates that action. This modification results in more accurate value estimates and improved stability across a variety of tasks. However, the method may temporarily revert to standard Q-learning behavior immediately after target network updates, slightly reducing its effectiveness during those periods.

3) Dueling DQN: Another important improvement is introduced by the Dueling DQN architecture (Wang et al., 2016), which decomposes the action-value function into two separate components: a state-value function and an advantage function that captures the relative importance of each action. This decomposition allows the model to generalize more effectively across actions, particularly in states where many actions have similar outcomes. As a result, learning becomes more efficient and stable. However, the decomposition introduces identifiability issues, requiring careful architectural design to ensure meaningful learning.

4) Rainbow DQN: Building on these advancements, the Rainbow DQN framework (Hessel et al., 2017) integrates multiple improvements into a single unified agent, including Double DQN, prioritized experience replay, dueling architectures, multi-step learning, distributional reinforcement learning, and noisy networks. By combining these techniques, Rainbow achieves state-of-the-art performance and significantly improves data efficiency. However, the contribution of individual components can vary across tasks, and not all improvements consistently enhance performance when combined.

Overall, value-based methods have evolved from simple tabular approaches to sophisticated deep learning-based frameworks capable of handling high-dimensional environments. These developments have played a crucial role in advancing the practical applicability of model-free reinforcement learning.

C. Policy-Based Methods

While value-based methods focus on estimating action-value functions, they often face challenges in continuous action spaces and stability. Policy-based methods represent an alternative to value-based approaches by directly parameterizing and optimizing the policy, rather than estimating action-value functions. This allows for more flexible learning, particularly in environments with continuous or high-dimensional action spaces. A foundational approach in this category is Policy Gradient Methods, which updates the policy parameters by following the gradient of expected cumulative reward.

1) Policy Gradient Methods: Policy gradient methods (Sutton et al., 1999) address the limitations of value-based approaches when combined with function approximators, where convergence guarantees are often difficult to establish. By directly optimizing the policy, these methods can operate effectively in complex environments using differentiable function approximators. A key contribution of policy gradient methods is the Policy Gradient Theorem, which provides a principled way to estimate gradients using sampled trajectories. However, these methods often suffer from high variance in gradient estimates, which can lead to slow and unstable learning. Additionally, they typically converge only to locally optimal policies rather than global optima.

2) Trust Region Policy Optimization (TRPO): To improve the stability of policy optimization, Trust Region Policy Optimization (TRPO) (Schulman et al., 2015) introduces a constrained optimization framework that limits how much the policy can change at each update. Specifically, TRPO enforces a constraint on the divergence between the old and new policies, ensuring that updates remain within a "trust region." This approach leads to more consistent performance and reduces the likelihood of performance collapse during training. However, TRPO is computationally expensive, as it requires solving a constrained optimization problem using second-order approximations such as the Fisher information matrix.

3) Proximal Policy Optimization (PPO): Building on these ideas, Proximal Policy Optimization (PPO) (Schulman et al., 2017) simplifies the optimization process while retaining the stability benefits of TRPO. Instead of enforcing a hard constraint, PPO uses a clipped objective function that restricts how much the policy can deviate from its previous version. This allows for multiple gradient updates using the same batch of data without causing destructive policy updates. As a result, PPO achieves a favorable balance between performance, stability, and computational efficiency. However, certain variants of PPO that rely on penalty-based formulations can be sensitive to the choice of hyperparameters.

Overall, policy-based methods provide a powerful framework for directly optimizing decision-making strategies, particularly in environments where value-based methods are difficult to apply. Their ability to handle continuous action spaces and integrate seamlessly with function approximation makes them a key component of modern reinforcement learning systems.

D. Actor-Critic Methods

To address the limitations of both value-based and policy-based approaches, hybrid methods have been developed. Actor-critic methods combine the strengths of value-based and policy-based approaches by maintaining both a value function (critic) and a parameterized policy (actor). The critic

estimates value functions to guide learning, while the actor updates the policy directly based on feedback from the critic. This framework enables more stable and efficient learning compared to using either value-based or policy-based methods alone.

1) Asynchronous Advantage Actor-Critic (A3C): To address instability and improve training efficiency, Asynchronous Advantage Actor-Critic (A3C) (Mnih et al., 2016) proposes a parallelized learning framework in which multiple agents interact with independent instances of the environment. By asynchronously updating shared model parameters, A3C reduces the correlation between training samples and stabilizes learning without relying on experience replay. This approach enables efficient on-policy learning and achieves strong performance with relatively low computational resources. However, the lack of experience replay can reduce data efficiency, as past experiences are not reused.

2) Deep Deterministic Policy Gradient (DDPG): An important development in this direction is Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2016), which extends actor-critic methods to continuous action spaces by learning deterministic policies using deep neural networks. DDPG introduces the use of experience replay and target networks within the actor-critic framework, improving stability when training with nonlinear function approximators. Despite its effectiveness, DDPG suffers from instability due to the accumulation of errors in value estimates, which can negatively impact policy learning.

3) Twin Delayed Deep Deterministic Policy Gradient (TD3): Another important advancement is Twin Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al., 2018), which specifically addresses overestimation bias in value estimates. TD3 introduces the use of two critic networks and updates the policy less frequently than the value function, resulting in more stable learning targets. By taking the minimum of two value estimates, TD3 reduces the likelihood of overestimation, improving both learning stability and final performance. However, this approach can introduce a slight underestimation bias in the value function.

4) Soft Actor-Critic (SAC): Further improvements in stability and sample efficiency are achieved by Soft Actor-Critic (SAC) (Haarnoja et al., 2018), an off-policy method that incorporates entropy maximization into the learning objective. By encouraging exploration through entropy regularization, SAC achieves more robust and consistent performance across different tasks. Additionally, its off-policy nature allows it to reuse past experiences, improving sample efficiency compared to on-policy methods. However, SAC can be sensitive to certain hyperparameters, such as the reward scaling factor, which must be tuned carefully for optimal performance.

Overall, actor-critic methods provide a flexible and powerful framework for reinforcement learning, particularly in continuous control tasks. By combining value estimation with direct policy optimization, these methods achieve a balance between stability, efficiency, and performance, making them widely used in modern reinforcement learning applications.

E. Modern Advances in Model-Free Reinforcement Learning

Despite significant progress, traditional model-free methods often require large amounts of interaction data and may be difficult to apply in real-world settings. Recent advances in model-free reinforcement learning have focused on addressing key limitations that hinder real-world applicability, particularly in settings where data is limited, expensive to collect, or cannot be generated through continuous interaction. Two major directions have emerged: offline reinforcement learning, which enables learning from fixed datasets, and data-efficient reinforcement learning, which improves learning under limited interaction.

1) Conservative Q-Learning (CQL): A central challenge in offline reinforcement learning is the issue of distributional shift, where the learned policy evaluates actions that are not present in the dataset, leading to inaccurate and often overestimated value functions. This problem is particularly critical in real-world domains such as robotics and recommendation systems, where collecting additional data through exploration may be costly or unsafe. To address this, Conservative Q-Learning (CQL) (Kumar et al., 2020)introduces a framework that directly regularizes the learned action-value function to produce conservative estimates. By explicitly penalizing the Q-values of out-of-distribution actions while reinforcing values on the observed data distribution, CQL ensures that the learned value function provides a lower-bound estimate of the true return. This approach eliminates the need to explicitly constrain the policy or model the behavior policy, distinguishing it from earlier offline RL methods. As a result, CQL demonstrates strong performance on complex datasets and high-dimensional tasks. However, it shares a key limitation with supervised learning approaches, namely susceptibility to overfitting, and lacks a straightforward mechanism for determining optimal stopping during training.

2) Implicit Q-Learning (IQL): Building on similar challenges, Implicit Q-Learning (IQL) (Kostrikov et al., 2022) proposes an alternative strategy for handling distributional shift without explicitly penalizing out-of-distribution actions. Instead of querying the value of unseen actions, IQL uses expectile regression to estimate the value of high-quality actions directly from the dataset. By fitting a state-value function to the upper expectiles of the action-value distribution, the method effectively captures the value of the best actions available within the data, while avoiding the instability caused by extrapolation. This design enables the algorithm to perform multi-step dynamic programming and even combine suboptimal trajectories to form improved policies. Compared to prior approaches, IQL is computationally efficient and relatively simple to implement, making it attractive for practical deployment. However, its performance is sensitive to the choice of the expectile parameter, which introduces a trade-off between approximation quality and optimization difficulty.

3) DrQ (Data-Regularized Q): In parallel, another line of work focuses on improving data efficiency in online reinforcement learning, particularly in high-dimensional environments such as those involving raw visual inputs. A key challenge in these settings is overfitting of neural network encoders when training with limited interaction data. DrQ (Data-Regularized

Q) (Yarats et al., 2021) addresses this issue by introducing a simple yet effective regularization strategy based on data augmentation. By applying random transformations to input images and averaging value estimates across multiple augmented versions, DrQ reduces variance in the learned Q-function and improves generalization. Unlike earlier approaches that rely on auxiliary losses or learned world models, DrQ achieves strong performance using only value-based learning combined with lightweight augmentations. This makes it computationally efficient and easy to integrate into existing pipelines. However, the method is constrained to transformations that preserve task semantics, limiting the range of augmentations that can be applied.

Overall, modern developments in model-free reinforcement learning emphasize robustness, stability, and efficient use of data. By addressing distributional shift in offline settings and improving generalization in data-limited environments, these approaches significantly expand the practical applicability of reinforcement learning in real-world scenarios.

V. Comparison of Model-Based and Model-Free Reinforcement Learning

Having surveyed the two paradigms in turn, we now return to the central organising question of this work: how do model-based and model-free reinforcement learning compare in practice, and when should a practitioner prefer one over the other? This section draws together evidence from the preceding algorithmic discussions and from broader surveys and empirical benchmarks [32], [33], [31], [34] to present a systematic comparison, followed by a discussion of the hybrid approaches that are increasingly blurring the boundary between the two families.

A. The Origin of the Trade-offs

The practical differences between model-based and model-free methods are not incidental; they follow directly from the structure of the Bellman optimality equation (6) introduced in Section II. Its right-hand side requires summing over next states weighted by transition probabilities $P(s' \mid s, a)$ and immediate rewards $R(s, a)$. Model-based methods evaluate this expectation through an explicit model, using planning to propagate information about many hypothetical actions per real interaction step. Model-free methods substitute this expectation with samples drawn from environment interaction, obtaining unbiased but high-variance estimates. This single algorithmic choice—exploit the expectation analytically, or approximate it empirically—gives rise to a predictable pattern of trade-offs that has been observed consistently across domains and across decades of research [32], [31].

B. Dimensions of Comparison

Table I summarises the principal differences between the two paradigms along nine dimensions synthesised from recent surveys [31], [34], [33]. The remainder of this section discusses the most consequential of these trade-offs in detail and interprets them in the light of the algorithms covered earlier.

C. Sample Efficiency versus Asymptotic Performance

The most consequential trade-off in the literature is the tension between *sample efficiency*—how much real-world interaction is required to reach a given level of performance—and *asymptotic performance*—the ceiling that a method can ultimately attain given unlimited data. Model-based methods excel at the former because a learned model can be queried without any additional environment interaction; each real sample implicitly informs many planned updates. Clavera et al. [3] report that model-based meta-policy optimisation matches the asymptotic performance of strong model-free baselines while using one to two orders of magnitude fewer samples on continuous control benchmarks. This advantage is decisive in domains where interaction is expensive, slow, or unsafe—notably physical robotics, where PETS [4] learns effective policies from only a small number of real trials.

Model-free methods pay the opposite bill. Canonical algorithms such as DQN [12] required on the order of tens of millions of frames to reach human-level play on the Atari benchmark, and even recent actor-critic methods such as SAC still demand substantial interaction budgets for high-dimensional continuous control. Their compensating advantage is robustness: because they never commit to a learned approximation of the dynamics, their estimates are in principle unbiased, and given sufficient data they can match the true optimum of the underlying MDP. Empirical comparisons confirm this pattern in both online [34] and offline [35] settings, where the relative ranking of model-based and model-free algorithms is seen to depend heavily on the available interaction budget.

D. Model Bias and Compounding Error

A closely related trade-off concerns robustness to approximation error. Because model-free methods learn values or policies directly from sampled transitions, their errors are bounded and local: a poor Q-estimate at one state-action pair does not corrupt estimates elsewhere unless explicitly backed up by further interaction. Model-based methods, by contrast, propagate errors multiplicatively through multi-step planning rollouts, an effect known as *compounding error*. A small misprediction of $P(s' \mid s, a)$ early in a rollout produces an incorrect downstream state distribution, which in turn yields increasingly unreliable value estimates. Moerland et al. [31] emphasise that addressing this problem—through probabilistic ensembles [4], meta-policy optimisation across an ensemble of learned dynamics [3], short-horizon rollouts, or Bayesian dynamics models—is the single largest open challenge for modern model-based reinforcement learning.

E. Computational and Structural Considerations

Beyond sample efficiency and bias, the two paradigms differ in where they spend computation. Model-free policies, once trained, are extremely cheap to deploy: action selection reduces to a single forward pass or table lookup. Model-based methods—particularly those using model-predictive control—often perform planning at every decision step, trading data efficiency for runtime cost. This makes model-free methods better

TABLE I
COMPARISON OF MODEL-BASED AND MODEL-FREE REINFORCEMENT LEARNING ACROSS NINE DIMENSIONS

Dimension	**Model-Based RL**	**Model-Free RL**
Environment model	Assumed or learned explicitly ($\hat{P}, \hat{R}$)	Not required; learned directly from interaction
Sample efficiency	High: one real transition can inform many planned updates	Low: each policy update typically requires many real transitions
Asymptotic performance	Upper-bounded by the learned model's accuracy (*model bias*)	Not model-limited; can in principle match the true optimum
Computational cost per decision	High: requires planning or rollouts at decision time (e.g. MPC)	Low: a single policy or Q-network forward pass
Sensitivity to model bias	High: small errors compound over multi-step rollouts	None; no model to be biased
Exploration mechanism	Can be uncertainty-driven using model disagreement or ensembles	Typically ϵ-greedy, parameter noise, or entropy regularisation
Scalability to high-dim. observations	Historically limited; improving with latent-space and probabilistic models	Strong with deep networks (e.g. DQN, SAC)
Suitability for continuous control	Strong (MPC, PETS, MB-MPO)	Strong (DDPG, TD3, SAC)
Representative algorithms	Value/policy iteration, Dyna, PETS, MB-MPO, AlphaZero	Q-learning, SARSA, DQN, PPO, SAC, DDPG, TD3

suited to real-time and embedded deployment, while model-based methods are favoured when training-time interaction, rather than inference-time compute, is the scarcer resource. Value-based model-free methods also scale gracefully to the high-dimensional observation spaces that dominate modern applications; Arulkumaran et al. [33] attribute much of deep RL's empirical success to this property, which model-based approaches have only recently begun to match through latent-space world models and probabilistic dynamics networks.

F. Convergence of the Two Paradigms

Although the preceding discussion treats model-based and model-free methods as distinct, the contemporary literature increasingly portrays them as endpoints of a continuum rather than a binary choice [31], [34]. *Hybrid* approaches combine model-based planning with model-free policy learning in order to inherit the sample efficiency of the former and the robustness of the latter. The Dyna framework [11] interleaves real and model-generated experience to update a Q-function; MB-MPO meta-optimises a model-free policy over an ensemble of learned dynamics models [3]; AlphaZero [8] uses Monte Carlo Tree Search over a known model to guide a deep policy-value network. Recent empirical benchmarks [35], [34] suggest that on many realistic problems the best-performing algorithms are neither purely model-free nor purely model-based, but hybrids that adapt their reliance on the learned model to local uncertainty in the dynamics. This trajectory suggests that future research will focus less on choosing between the two paradigms and more on learning *when* to trust a model and *how* to fall back gracefully on direct interaction when the model fails.

G. Practical Guidance

Synthesising these observations, the paradigm choice for a given application can be reduced to a small set of practical questions. Model-based methods are particularly well-suited when (i) environment interaction is expensive, risky, or slow—as in physical robotics, clinical treatment, or industrial control; (ii) the underlying dynamics are smooth and representable with modest function-approximator capacity; or (iii) interpretability and the ability to plan over an explicit model are themselves valuable. Model-free methods tend to be preferred when (i) a high-fidelity simulator is available and samples are effectively free, as in game-playing and many simulated control benchmarks; (ii) the observation space is very high-dimensional, such as raw visual input; or (iii) runtime inference must be fast enough for real-time control without planning overhead. When neither regime is clearly dominant—which describes most real-world problems—hybrid methods that learn a model but use it selectively, rather than committing fully to either paradigm, are increasingly the default recommendation in the literature [31], [34].

VI. APPLICATIONS OF REINFORCEMENT LEARNING

Reinforcement learning has moved from an academic curiosity to a deployed technology over the past decade, with applications spanning games, robotics, autonomous driving, healthcare, and industrial operations. This section surveys representative uses across these domains, drawing both on the algorithmic discussion of the preceding sections and on dedicated application-focused surveys [33], [37], [36]. A complementary discussion of settings in which model-based methods specifically have been influential—including strategic gaming, precise robotic manipulation, and computational neuroscience—is given in Section **??**. Our emphasis here is on drawing out the *pattern* of which paradigm has proved dominant in which setting, and why.

A. Games and Strategic Decision-Making

Games have long served as the primary testbed of reinforcement learning research, and they offer the clearest illustration of the paradigm divide. Mnih et al. [12] demonstrated that a model-free deep Q-network could reach human-level

performance on 49 Atari 2600 games directly from pixel inputs, establishing deep model-free RL as a viable paradigm for high-dimensional decision-making. At the other end of the spectrum, AlphaGo Zero [8] combined the exact known model of the game of Go with deep reinforcement learning and Monte Carlo Tree Search, attaining superhuman play without any human demonstrations. Subsequent systems such as OpenAI Five and AlphaStar extended these ideas to real-time strategy games with much larger action spaces, relying on model-free policy-gradient methods at massive scale [33]. The pattern is clear: where an exact or near-exact model is available—as in board games—planning-based approaches dominate; where the environment is intractable to model but cheap to simulate—as in video games—model-free methods win.

B. Robotics and Continuous Control

Robotics is the domain in which the sample-efficiency advantage of model-based RL has had the greatest practical impact. Collecting real-world interaction data on a physical robot is slow, costly, and risks hardware damage, which makes the one-to-two orders-of-magnitude sample-efficiency advantage reported for methods such as PETS [4] and MB-MPO [3] decisive. Finn and Levine [9] showed that learned video-prediction models can support real-robot manipulation from raw visual input, and more recent latent-space world models have extended these ideas to locomotion and dexterous control. Where training can be conducted entirely in simulation, however, model-free actor-critic methods such as SAC and TD3 are competitive and often simpler to tune, and they remain the workhorse of standard simulated locomotion benchmarks [34].

C. Autonomous Driving

Autonomous driving presents a hybrid of the robotics and gaming cases: physical deployment is unsafe, but high-fidelity simulators such as CARLA make synthetic interaction effectively free. Kiran et al. [37] survey the use of deep RL across the autonomous-driving stack, including lane-keeping, merging, overtaking, and intersection negotiation. Model-free actor-critic methods—DDPG, SAC, and PPO—are the most common choice in this literature, typically trained in simulation and then transferred to real vehicles via the so-called *sim-to-real* pipeline. Model-based and hybrid approaches are increasingly explored for their ability to incorporate explicit safety constraints at planning time, which is difficult to enforce in a purely model-free policy.

D. Healthcare and Clinical Decision-Making

Healthcare sits at the opposite extreme of the simulation-availability spectrum: interaction with patients is neither repeatable nor risk-free, and no accurate simulator of human physiology exists. Yu et al. [36] survey more than a hundred studies applying RL to dynamic treatment regimes, intensive-care decision support, sepsis management, and drug dosing, observing that offline and model-based approaches dominate the field. Recent offline model-free methods such as Conservative Q-Learning and Implicit Q-Learning, discussed in Section **??**, directly target this setting by learning from fixed datasets of past patient records without requiring further exploration. RL has also contributed to cognitive-neuroscience modelling, including explanations of dopaminergic dysfunction in Parkinson's disease as a failure of model-based value learning [10].

E. Energy Systems and Industrial Operations

Energy management, building climate control, and industrial operations are domains in which partial physical models of the system—load forecasts, weather predictions, equipment dynamics—are often available, making them natural candidates for model-based approaches. Yao et al. [7] report that a model-based RL scheduler for microgrid dispatch yields substantial cost savings over heuristic controllers by exploiting such structure. Similar gains have been reported for HVAC control, data-centre cooling, and supply-chain optimisation, where the combination of learned and physics-based models enables planning over hours or days without the expense of purely data-driven exploration.

F. Cross-Cutting Observations

Taken together, these applications reveal a consistent organising principle: the choice of paradigm is driven less by the specific task than by two properties of the environment. The first is the cost of a single interaction—high in robotics, healthcare, and energy systems, but negligible in simulated games. The second is the availability of an accurate forward model—present in board games, partial in energy systems, and essentially absent in complex video games and driving. Model-based methods dominate when interaction is expensive *or* a model is available; model-free methods dominate when simulated interaction is cheap *and* no accurate model is available; hybrid approaches are emerging as the best compromise when neither condition is fully met [31], [34]. This pattern, rather than any particular algorithm, is the most durable lesson the applications literature offers for practitioners selecting an RL approach for a new domain.

VII. Challenges and Future Directions

Despite the empirical successes surveyed in the preceding sections, reinforcement learning remains an open research agenda rather than a mature engineering discipline. The challenges discussed below, drawn from recent surveys [31], [33], [34] and from the domain-specific literature cited in Section VI, cut across the model-based/model-free distinction and together define the frontier of current research.

A. Sample Efficiency and the Cost of Interaction

Sample efficiency remains the single dominant open problem in reinforcement learning. Although model-based methods have been shown to require one to two orders of magnitude fewer interactions than their model-free counterparts on continuous control benchmarks [3], [4], both paradigms still lag far behind biological learning in settings that demand

exploration under sparse feedback. The practical consequences are most acute in domains where interaction is inherently costly—physical robotics, clinical decision-making, and autonomous driving—and where the gap between simulator and reality further limits the transferability of cheaply obtained experience. Offline reinforcement learning [35] and model-based data augmentation are two active research directions that seek to close this gap; both remain constrained by issues of distributional shift and compounding model error, respectively.

B. Stability and Convergence in Deep RL

Deep reinforcement learning introduces instability challenges that the tabular algorithms of Section II do not face. The combination of function approximation, bootstrapping, and off-policy updates—Sutton and Barto's "deadly triad" [11]—can produce divergence even on problems where classical analysis guarantees convergence. Stabilising techniques such as experience replay, target networks, and trust-region constraints, discussed in Section **??**, have largely addressed these issues in practice, but they do so through engineering heuristics rather than principled theory. Model-based methods add a further source of instability through compounding error in learned dynamics, which remains the central obstacle to scaling MBRL to long-horizon tasks [31].

C. Safety, Robustness, and Real-World Deployment

Deploying RL agents in safety-critical domains demands guarantees that current algorithms do not naturally provide. Kiran et al. [37] identify safety as the foremost obstacle to real-world autonomous driving, and similar concerns dominate the healthcare literature, where Yu et al. [36] emphasise the difficulty of validating learned policies before clinical use. Related challenges include the *reality gap*—the mismatch between simulator and real-world dynamics that degrades sim-to-real transfer—and *distribution shift*, where a policy trained on one data distribution fails when deployed on another. Model-based methods have a potential advantage here because their learned dynamics can be inspected and constrained, but whether this advantage survives the compounding-error problem is itself an open question.

D. Exploration in Sparse-Reward and Long-Horizon Problems

Effective exploration remains one of the oldest unsolved problems in reinforcement learning [32]. The ϵ-greedy and entropy-based strategies used in most modern model-free algorithms scale poorly to environments in which reward is extremely sparse or delayed by thousands of time steps. Model-based methods can, in principle, direct exploration toward states of high epistemic uncertainty using model ensembles or Bayesian posteriors [4], but realising this potential in high-dimensional observation spaces is still an active research area. Complementary directions receiving increasing attention include curiosity-driven intrinsic motivation, hierarchical reinforcement learning to decompose long-horizon tasks into manageable sub-problems, and goal-conditioned policies.

E. Interpretability and Trustworthy RL

As reinforcement learning moves from benchmark environments to deployed systems, interpretability and trustworthiness become practical requirements rather than aesthetic preferences. A trained deep Q-network or actor-critic policy is effectively a black box; neither the basis for individual decisions nor the policy's failure modes are easily accessible to a human operator [34]. Model-based approaches offer partial relief through the inspectability of their learned dynamics, but do not resolve the issue for the policy itself. Explainable RL, formal verification of learned policies, and human-in-the-loop training are all active research directions motivated by this gap.

F. Toward Foundation Models and Unified Paradigms

Two longer-term trends are reshaping the field. The first is the continuing convergence of model-based and model-free methods into hybrid architectures, discussed in Section V—a trend that recent empirical comparisons [35], [34] suggest will accelerate. The second is the integration of reinforcement learning with large pre-trained foundation models: large language models have already transformed policy specification through reinforcement learning from human feedback, and pre-trained world models are beginning to play an analogous role for model-based control. Together these trends point toward a future in which the rigid model-based/model-free taxonomy of the past two decades gives way to more flexible architectures that combine learned world knowledge, sampled interaction, and explicit planning in task-dependent proportions.

VIII. Conclusion

This paper has presented a comparative survey of model-based and model-free reinforcement learning, two paradigms that share the Markov Decision Process formalism but differ in whether they exploit an explicit model of the environment or learn directly from interaction. We reviewed the principal algorithms in each family, from classical dynamic programming and tabular temporal-difference methods to modern deep variants including PETS, MB-MPO, DQN, Rainbow, PPO, and SAC.

Across nine dimensions of comparison, the two paradigms exhibit complementary strengths. Model-based methods achieve substantially higher sample efficiency and support explicit planning, at the cost of susceptibility to compounding errors in learned dynamics. Model-free methods, while less data-efficient, remain unbiased at scale and generalise gracefully to high-dimensional observations. We argued that these differences follow directly from the structure of the Bellman optimality equation, and that the choice between paradigms in a given application is governed less by the specific task than by the cost of environment interaction and the availability of an accurate forward model.

The most important trend in the recent literature is the convergence of the two paradigms into hybrid architectures that learn a model but use it selectively. Systems such as AlphaGo Zero, MB-MPO, and modern Dyna variants already combine the sample efficiency of model-based planning with

the robustness of model-free learning, and empirical benchmarks suggest this hybrid direction will dominate practical deployments going forward. Looking further ahead, the integration of reinforcement learning with large pre-trained foundation models is poised to reshape the field again, offering the prospect of agents that combine learned world knowledge, sampled interaction, and explicit planning in task-dependent proportions. The rigid model-based/model-free taxonomy that has organised the past two decades of research is, we expect, giving way to a more flexible synthesis.

Appendix A
Proof of the First Zonklar Equation

Appendix one text goes here.

Appendix B

Appendix two text goes here.

Acknowledgment

The authors would like to thank...

References

[1] F. J. Yi, W. L. Fu, and H. Liang, "Model-based reinforcement learning: A survey," in *Proc. 18th Int. Conf. Electron. Bus. (ICEB)*, Guilin, China, 2018, pp. 421–429.

[2] C.-V. Pal and F. Leon, "Brief Survey of Model-Based Reinforcement Learning Techniques," in *Proc. 24th Int. Conf. Syst. Theory, Control Comput. (ICSTCC)*, Sinaia, Romania, 2020, pp. 92–97.

[3] I. Clavera et al., "Model-based reinforcement learning via meta-policy optimization," *arXiv preprint arXiv:1809.05214*, 2018.

[4] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[5] T. M. Moerland, J. Broekens, and C. M. Jonker, "Learning multimodal transition dynamics for model-based reinforcement learning," *arXiv preprint arXiv:1705.00470*, 2017.

[6] Ç. Yıldız, M. Heinonen, and H. Lähdesmäki, "Continuous-time model-based reinforcement learning with Bayesian neural ordinary differential equations," *arXiv preprint arXiv:2102.04778*, 2021.

[7] J. Yao, J. Xu, N. Zhang, and Y. Guan, "Model-Based Reinforcement Learning Method for Microgrid Optimization Scheduling," *Sustainability*, vol. 15, no. 12, p. 9235, 2023.

[8] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[9] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2016.

[10] M. E. Sharp et al., "Dopamine restores model-based learning deficits in patients with Parkinson's disease," *Biological Psychiatry*, vol. 77, no. 10, pp. 903–912, 2015.

[11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[12] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[13] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2005.

[14] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, pp. 9–44, 1988.

[15] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[16] G. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Technical Report CUED/F-INFENG/TR 166, 1994.

[17] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[18] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," *arXiv preprint arXiv:1509.06461*, 2015.

[19] Z. Wang et al., "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, 2016.

[20] M. Hessel et al., "Rainbow: Combining improvements in deep reinforcement learning," *arXiv preprint arXiv:1710.02298*, 2017.

[21] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12, 1999.

[22] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," *arXiv preprint arXiv:1502.05477*, 2015.

[23] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[24] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," *arXiv preprint arXiv:1602.01783*, 2016.

[25] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2016.

[26] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," *arXiv preprint arXiv:1802.09477*, 2018.

[27] T. Haarnoja et al., "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *arXiv preprint arXiv:1801.01290*, 2018.

[28] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," *arXiv preprint arXiv:2006.04779*, 2020.

[29] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," *arXiv preprint arXiv:2110.06169*, 2022.

[30] D. Yarats, I. Kostrikov, and R. Fergus, "Image augmentation is all you need: Regularizing deep reinforcement learning from pixels," *arXiv preprint arXiv:2004.13649*, 2021.

[31] T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker, "Model-based reinforcement learning: A survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 1, pp. 1–118, 2023.

[32] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

[33] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A brief survey of deep reinforcement learning," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, Nov. 2017.

[34] M. Ghasemi, A. H. Moosavi, and D. Ebrahimi, "A comprehensive survey of reinforcement learning: From algorithms to practical challenges," *arXiv preprint arXiv:2411.18892*, 2024.

[35] P. Swazinna, S. Udluft, D. Hein, and T. A. Runkler, "Comparing model-free and model-based algorithms for offline reinforcement learning," *arXiv preprint arXiv:2201.05433*, 2022.

[36] C. Yu, J. Liu, S. Nemati, and G. Yin, "Reinforcement learning in healthcare: A survey," *ACM Computing Surveys*, vol. 55, no. 1, Art. no. 5, Dec. 2021.

[37] B. R. Kiran et al., "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, Jun. 2022.